

ACU 深度学习方向考核

一、考核介绍

- 1.本次考核存在一定难度，所有基础必须全部完成,附加项目不作要求。
- 2.评判标准主要基于对已完成项目的掌握程度，拓展程度，态度。
- 3.若自身有其他感兴趣的方向，并进行了学习与实验，也可一并提交，作为加分项。
- 4.考核周期约为一个月，具体时间后续招新群中通知。

二、注意事项

1.推荐开发工具

- (1) IDE : Pycharm、Visual Studio Code
- (2) 包管理与环境管理工具： Anaconda (强烈推荐)

2.项目参考资料

(1) 环境配置

- ①Anaconda 环境配置：<https://www.cnblogs.com/xiaoxiaojie/p/14470905.html>
- ②Cuda 环境配置（根据自身条件选装）：<https://zhuanlan.zhihu.com/p/358737417>

(2) 基础知识学习（推荐 up 主）

Python 基础

- ①视频教程：【黑马程序员 python 零基础全套教程，8 天 python 从入门到精通，学 python 看这套就够了- 哔哩哔哩】<https://b23.tv/2arQVSG>
- ②文字教程：菜鸟教程 python 基础：<https://www.runoob.com/python/python-tutorial.html>

Pytorch 基础

- ①李沐（建议有一定基础的同学跟随学习）
<https://space.bilibili.com/1567748478/channel/seriesdetail?sid=358497>
- ②李宏毅（适合 0 基础入门）<https://www.bilibili.com/video/BV1hE411t7RN/>

③【PyTorch 深度学习快速入门教程（绝对通俗易懂！）【小土堆】】

<https://www.bilibili.com/video/BV1hE411t7RN/>

④【《PyTorch 深度学习实践》完结合集】<https://www.bilibili.com/video/BV1Y7411d7Ys/>

⑤【00 预告【动手学深度学习 v2】】

三、总文件提交要求

1.提交时间预计为五月中旬底，具体时间请关注招新群。具体答辩形式、答辩时间、答辩地点之后再进行通知。

2.将所有文件打一个包，压缩发送至负责人邮箱：1563584846@qq.com，压缩包以“深度学习+姓名+学号”命名。

PS： 每一个项目都有各自的提交要求，请按照相关要求提交文件。除此之外，还需要提交相关项目的答辩 PPT(可以几个项目汇总做一个 PPT，也可以每一个项目做一个 PPT，PPT 不必考虑美化，主要是内容呈现)。

基础项目一：基于轻量化 ResNeXt 的图像分类

一、项目说明

图像分类是深度学习在计算机视觉领域的基础核心任务，目标是根据图像的内容，为其分配一个或多个预设的类别标签（比如判断一张图是“猫”“狗”还是“汽车”）。它是目标检测、语义分割、图像生成等复杂视觉任务的技术基石。

在此项目中，你将学习并实现一种更先进的网络架构——ResNeXt。ResNeXt 是 ResNet 的改进版本，由 Saining Xie、何恺明等人于 2017 年提出（论文：Aggregated Residual Transformations for Deep Neural Networks, CVPR 2017）。其核心创新在于引入了“基数”这一新维度，即在残差块内部采用分组卷积，通过并行多个相同拓扑结构的变换路径来聚合特征。

本项目要求你手动实现一个轻量化 ResNeXt 网络，在给定的 101 类图像数据集上完成分类任务的全流程实践。通过本项目，你将掌握 ResNeXt 的核心设计思想，理解分组卷积与基数的概念，并能够与 ResNet 进行对比分析。

项目配备包含 101 类图像的数据集，学习者将在该数据集上完成从模型搭建、训练迭代到性能评估的完整实践，切实提升对卷积神经网络的应用与驾驭能力。

1.数据规模

10686 张图像（101 个类别，每个类别的图像张数不等）

2.划分比例（按 7:2:1 划分）

训练集：6941 张（约 70%）

验证集：2379 张（约 20%）

测试集：1366 张（约 10%）

3. 图像特点

已统一调整为 224×224 像素（适配网络输入）

以下是本次项目所需要实现的网络结构表：

层级	操作	输入尺寸	核大小/步	分	输出尺寸
----	----	------	-------	---	------

			长/填充	组数	
输入图像	-	batchx3x224x224	-	-	batchx3x224x224
Stem Conv	Conv2d + BN + ReLU	batchx3x224x224	7x7,s=2,p=3	1	batchx64x112x112
Stem Pool	MaxPool2d	batchx64x112x112	3x3,s=2,p=1	-	batchx64x56x56
Stage1(x2)	ResNeXt Block (无下采样)	batchx64x56x56	见下方说明	8	batchx256x56x56
Stage2(x2)	ResNeXt Block (第1块下采样)	batchx256x56x56	见下方说明	8	batchx512x28x28
Stage3(x2)	ResNeXt Block (第1块下采样)	batchx512x28x28	见下方说明	8	batchx1024x14x14
Stage4(x2)	ResNeXt Block (第1块下采样)	batchx1024x14x14	见下方说明	8	batchx2048x7x7
Global Pool	AdaptiveAvgPool2d	batchx2048x7x7	-	-	batchx2048x1x1
展平	Flatten	batchx2048x1x1	-	-	batchx2048
Classifier	Linear	batchx2048	-	-	batchx101

4.ResNeXt Block 内部结构说明 (核心)

每个 ResNeXt Block 由三个卷积层组成(均后接 BatchNorm 和 ReLU),并带有残差短接:

(1) 1×1 卷积(降维): 输入通道 $\rightarrow$ 中间通道 (中间通道 = 基数 $\times$ 每分支通道)

(2) 3×3 分组卷积: groups = 基数, 步长根据是否下采样取 1 或 2

(3) 1×1 卷积(升维): 中间通道 $\rightarrow$ 输出通道

(4) 短接: 若输入输出维度相同, 使用恒等映射; 若下采样或通道数变化, 使用 1×1 卷积(步长匹配)

(5) 各阶段详细参数:

阶段	输入通道	中间通道(总)	基数	每分支通道	输出通道	块数量	下采样(第1块)
Stage1	64	128	8	16	256	2	否
Stage2	256	256	8	32	512	2	是(s=2)
Stage3	512	512	8	64	1024	2	是(s=2)
Stage4	1024	1024	8	128	2048	2	是(s=2)

(6) 网络结构流程:

输入 → 卷积(7×7) → BN → ReLU → 最大池化 → ResNeXt 块(Stage1×2) → ResNeXt 块(Stage2×2) → ResNeXt 块(Stage3×2) → ResNeXt 块(Stage4×2) → 全局平均池化 → 全连接层 → 输出（预测类别）

以下为训练配置项（各参数可根据自身情况修改）：

配置项	参数值
Batch Size	32
Epochs	80
Learning Rate	0.01
Optimizer	SGD（momentum=0.9, weight_decay=1e-4）
Loss Function	CrossEntropyLoss
Image Size	224×224×3
Train:Val:Test	7:2:1（已划分）

二、考核目的

本次图像分类任务，主要考察大家对于深度学习整个过程的了解程度，重点要求掌握数据集的预处理、分组卷积与残差结构的实现、Python 语法以及 PyTorch 框架的使用。同时，希望你能对比 ResNeXt 与 ResNet 的异同，理解“基数”这一维度对模型性能的影响。

三、考核要求

实现正确分类，具体而言即给出一张输入图像（格式为 RGB），通过网络预测，能够得到输出结果。例如传入一张猫的 RGB 图片，网络预测的返回值为“cat”。

1. 框架选择

使用自己熟悉的一款深度学习框架，例如 PyTorch、TensorFlow 和 Paddle 等。如果没有相关学习经历，建议使用 PyTorch 框架。

2. 数据预处理

必须使用考核项目中给出的数据集，数据集已进行具体划分，可进行解压、数据去噪和数据增强等操作。

(1) train: 程序用于训练模型，包括显示训练进度、记录损失函数变化过程、打印训练日志并保存权值文件。

(2) val: 程序用于在验证集上评估训练后的模型，以便在训练过程中调整超参数和检查模型性能。可以通过划分给定训练集的一部分作为验证集来进行验证。

(3) test: 程序用于在测试集上评估训练好的模型，以得到模型在未知数据上的泛化性能。

3.测试： 请详细说明你划分数据集的文件夹格式，保证测试程序能正确运行，我们将于考核结束后在测试集上运行你的模型，获取准确率，作为评估结果的一部分。

4.提交：

(1) 完整的源代码 (.py 或 ipynb, 不交数据集)

(2) 训练权值文件 (best 和 last 的 2 个权值文件即可)、训练日志 (txt 格式保存即可) 及图像化结果

(3) 说明文档 (主要是对于项目的介绍, 与在学习这一领域的具体想法, 着重谈自己的学习思考, 字数无强制要求)。说明文档中必须包含以下内容:

①解释 ResNeXt 的核心思想: 什么是基数? 分组卷积如何实现? 为什么增加基数比增加深度或宽度更有效?

②对比 ResNeXt 与 ResNet 的结构差异 (可附表格或图示)。

③分组卷积代码实现的关键点 (groups 参数的使用, 输入输出通道与基数的整除关系)。

④训练过程中遇到的问题及解决方法。

注： 禁止抄袭其他参考同学的代码，提交内容如有雷同，则自动退出本次考核。严禁直接调用 torchvision.models.resnext 等预定义模型，必须手动实现分组卷积残差块。

四、参考资料

1.实战教程

①猫狗分类实战教程: <https://zhuanlan.zhihu.com/p/388676784>

②保存日志文件教程: https://blog.csdn.net/qq_19168521/article/details/115704185

③保存权重文件以及测试时权重文件的读取:
<https://blog.csdn.net/tuanzi2809/article/details/112383990>

2.ResNeXt 专项资料

(1) 原始论文: Aggregated Residual Transformations for Deep Neural Networks
(<https://arxiv.org/abs/1611.05431>)

(2) PyTorch 官方实现参考 (仅用于理解结构, 不可直接复制):
torchvision.models.resnext50_32x4d

(3) 讲解视频：B 站搜索“ResNeXt 原理讲解”或“分组卷积”

五、提示

1.对于硬件条件有限的同学，可以尝试使用 Autodl、Colab 和 Kaggle 等云计算平台进行模型训练。

2.重点在于掌握学习过程和深入理解基本概念，而非单纯地追求精度。评判标准主要侧重于基础知识的掌握、实际运用以及清晰地解释所使用方法的原理。如果不能满足这些要求，反而可能会直接导致不合格。

3.鼓励深入研究相关论文（ResNeXt 原文），以便更好地理解原理和技术。

4.如果训练出的模型效果不佳，本题暂时无需考虑，请移至下一题。

5.深度学习代码推荐结构：

(1) mydataset.py: 负责处理数据集加载、预处理和划分

(2) model.py: 定义你的神经网络模型（ResNeXt）

(3) train.py: 负责训练模型，在验证集上评估模型

(4) test.py: 负责在测试集上运行模型

基础项目二

一、考核目的

在项目一（基于 ResNeXt 的图像分类）的基础上对模型进行改进，也可以更换所使用的模型，旨在提升模型在验证集上的表现。

将基础项目一中的改进单独写入一个说明文档(即考核报告)中,主要体现做出了哪些改动,改动效果如何,为什么这样做(原理)。

此项目不以绝对准确率高低为唯一标准，而重点考察：

1.改进动机是否源于对模型行为或数据特性的观察；

2.改进方法是否符合深度学习基本原理；

3.改进效果是否经过严谨验证与对比分析；

4.自身对改进方法原理及改进原因的理解。

二、改进方向提示

注：以下只是改进方向的提示，并不一定要完全按照下述提示改进网络。

1. 合理的实验设计过程

例如，结合数据集特点采用有效的数据预处理方法；尝试更合理的数据增强技巧；调整训练过程中的超参数设置与训练策略，并分析其对实验结果的影响；进行多组对比实验和消融实验，根据自己的理解对实验结果得出结论。

在实验过程中，应尽量基于项目一训练中观察到的问题进行改进，例如过拟合、欠拟合、收敛速度较慢、训练不稳定、类别间混淆明显等，并围绕这些现象设计改进方案。

对于 ResNeXt，还可以关注不同结构设置下模型容量、训练效率与分类效果之间的关系，并尝试从分组卷积和特征聚合的角度理解实验现象。

2. 改进网络结构

可以围绕项目一中手动实现的 ResNeXt 网络进行调整，例如结合 ResNeXt 中分组卷积与残差连接的特点，对残差块内部结构进行修改；尝试使用 Dropout、MaxPooling、BN、Residual Connection、Attention 等 tricks 对网络进行调整；或对网络深度、宽度以及分组方式进行适当变化，并完成相应的对比实验。

也可以结合自己对 ResNeXt 的理解，分析“基数”这一维度对模型表达能力的影响，思考不同分组设置、不同通道分配方式以及特征融合方式可能带来的变化。

在完成对比实验时，请结合自己的理解解释实验中的变化及原因，例如结构调整对特征提取能力、参数规模、训练稳定性和泛化性能可能造成的影响。

（并非一定要提高网络性能，而是要能解释实验结果。如果网络有所改进，但无法正确理解所采用的方法并解释其原因，将会是扣分项。）

3. 复现经典网络或使用其中部分结构

如 ResNet、VGG、AlexNet 等经典分类网络，或借鉴其中部分结构思想，与项目一中的 ResNeXt 进行对比分析。

也可以在保证能够正确解释网络结构与原理的前提下，参考经典网络中的某些模块设计，对现有模型进行适当替换或组合，并思考其与 ResNeXt 原有结构之间的差异。

要求确保在不爆内存的情况下使用服务器（如自己的笔记本）进行复现，并在相应的代码文件、考核报告和 PPT 中对网络进行正确解释（否则将成为减分项）。

注：本项目仍以经典图像分类任务为主，应围绕分类模型结构展开改进与比较。

三、提交要求

- 1.改进后的完整源代码（.py 或 ipynb）（需要标出改动位置，不交数据集）
- 2.改进后的训练权值文件（best 和 last 的 2 个权值文件即可）、训练日志（txt 格式保存即可）及图像化结果
- 3.改进前后验证集 acc 对比表格、训练/验证 loss 与 acc 曲线叠加图、关键超参数设置说明（核心材料）
- 4.说明文档（包括观察到的问题，如过拟合、收敛慢、训练震荡等等、选择的改进方法、预期效果与实际结果）

注：重点在于思考的过程，对改进方法原因及原理的理解。

附加项目一：理解 RT-DETR 核心思想

一、考核目的

学习如何基于前沿研究论文，动手实现一个先进的深度学习模型。本次考核旨在考察对现代目标检测模型全流程的理解与动手能力，重点要求掌握数据集的定制化预处理（掩码转边界框）、Transformer 检测模型（DETR）的基础结构、以及根据论文理解核心模块的工程能力。

二、数据集

请下载 crowdhuman 数据集，鉴于该数据集较大，需要自行从中抽取子集进行训练，自己规模约 3000 张图像左右。



网址如下：

<https://www.kaggle.com/datasets/loctran0941/crowdhuman?resource=download>

三、任务要求

1. 最终模型需输入一张图片，输出图中物体的边界框及类别。项目的核心考核点在于对 RT-DETR 论文中核心创新的代码级体现。重点理解如何对高层特征（S5）进行尺度内交互（AIFI），以及如何通过卷积实现跨尺度融合（CCFM）。
2. 数据预处理：必须使用给定的数据集。需要同学们自己编写脚本。确保图像增强与边界框坐标同步更新，防止训练震荡。
3. 日志：训练过程中，要求有清晰的日志输出（例如使用 `print` 或 `logging` 库）。同时也要对损失函数有所了解，主要是分类损失、定位损失、以及匈牙利算法处理二分图匹配问题，尝试解释 DETR 为何能取消 NMS。
4. 测试：必须从原始数据集中划分出一部分作为测试集（不参与训练），用于最终的模型评估和结果可视化。
5. 不要直接使用 Ultralytics 中的封装版本，建议从 `github` 上获取源码进行训练。
6. 加分项：尝试自己搭建 RT-DETR 的三大核心组件 Backbone、Hybrid Encoder 和 Transformer Decoder。
7. 加分项：特征图在进入 AIFI 模块前后的维度变化，以及 CCFM 模块如何通过 RepConv 或 Fusion 实现跨尺度信息交换。
8. 加分项，了解 CLIP 多模态双塔模型，思考若要将 CLIP 应用到 RTDETR 中可以怎么设计，可以阅读相关论文，写阅读报告；也可以发挥自己的思考，提出自己的方案

四、提交内容

1. 可运行代码：一个包含所有模块（数据处理、模型、训练、推理）的完整项目文件夹。
2. 最优权重：训练好的最佳模型权重文件（.pth 或 .pt）。

3.简短报告（包括但不限于下面部分）： 怎样进行数据处理，核心实现是什么。

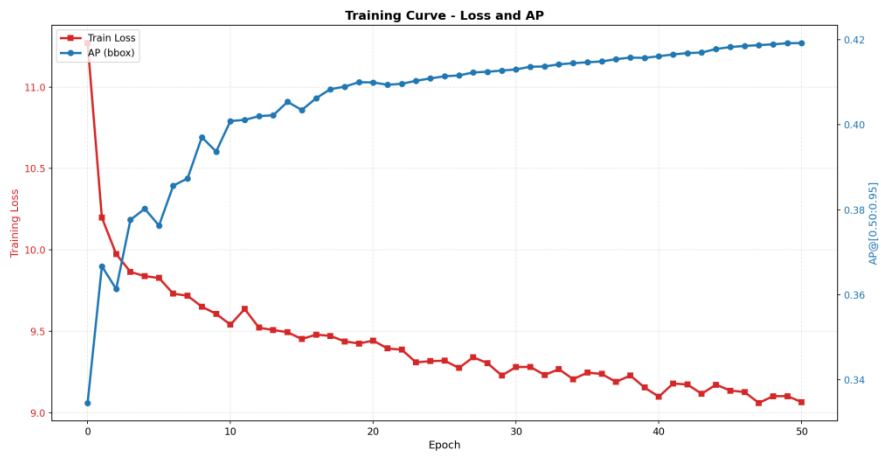
- (1) 可视化样例：附上 5~10 张带有预测结果的图片。
- (2) 误差分析：观察可视化结果，分析模型的优缺点，例如在何种场景下（如小物体、物体 遮挡、背景复杂）表现得好或不好，并尝试与论文中的分析联系起来。
- (3) 论文阅读情况（笔记、理解、过程多个维度）。 论文关键创新点理解情况。在项目基础上是否有自己的新理解，比如可以将模块独立出来做个小 demo 测试下。（不作要求）

五、常见坑

- 1.坐标格式：注意边界框坐标格式的统一，是 [x_min,y_min,x_max,y_max] 还是 [center_x, center_y, width, height]? 在计算损失和进行数据增强时要尤其小心。
- 2.维度对齐：在模型中，特征图的维度、Transformer 的输入输出维度、查询的维度等必须严格对齐，否则会频繁报错。
- 3.匈牙利匹配调试：这部分逻辑较为复杂，如果损失不下降，很可能是代价矩阵的计算或匹配 逻辑出了问题，需要耐心调试。
- 4.显存管理：DETR 类模型通常需要较大显存。如果遇到 OOM（OutofMemory）错误，请尝试减小 batch_size

六、训练效果基线

本结果为使用参数量较小 RTEDETR-resnet18 模型加载 coco 预训练权重，并在规模为 4500 的 crowdhuman 子集上微调并验证的结果，改良的训练精度不得低于该基线，有特殊情况请单独说明



附加项目二：基于 QLoRA 的大语言模型轻量化指令微调

一、考核目的

本次考核旨在考察对大语言模型轻量化微调核心技术的理解与工程落地能力，重点要求掌握 QLoRA 的核心创新原理、大模型有监督指令微调（SFT）的全流程实现、低资源环境下大模型训练与推理的工程化方法。同时，引导学习者深入理解大模型微调的底层逻辑，对比不同微调方案的优劣，建立对大模型定制化落地的完整认知。

具体考核目标如下：

1. 深入理解 QLoRA 的核心设计思想，对比全参数微调、标准 LoRA 的核心差异，掌握 4 位量化、NF4 数据类型、双量化等关键技术的原理与作用。
2. 完整掌握大模型指令微调的全流程：包括数据集预处理、环境配置、量化模型加载、QLoRA 适配器搭建、模型训练、权重合并、推理验证与效果评估的端到端实现。
3. 提升低资源环境下的大模型工程能力，解决微调过程中的显存优化、模型收敛、效果对齐等实际工程问题，实现单卡可运行的轻量化微调方案。
4. 理解指令微调的核心逻辑，能够基于特定场景完成定制化微调，定性与定量分析微调前后模型的能力变化，建立严谨的实验与分析思维。

二、数据与代码

1. 数据集要求

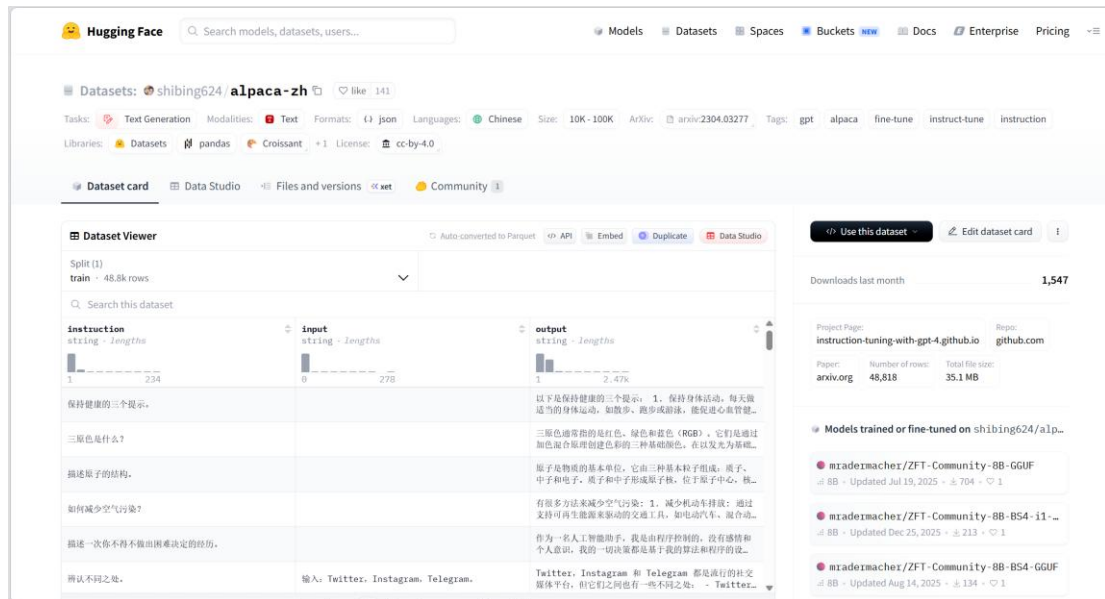
必须使用开源中文指令微调数据集，允许抽取小规模子集完成训练，重点是打通完整流程，不以数据集规模和模型指标为唯一评判标准，推荐数据集如下：

核心推荐：Alpaca-GPT4 - 中文数据集（精简版），下载地址：

<https://huggingface.co/datasets/shibing624/alpaca-zh>

备选数据集：Firefly 中文指令数据集、SchoolGPT 校园场景指令数据集、HuatuogPT 医疗科普指令数据集，可根据自身兴趣选择垂直领域数据集

数据规模要求：从开源数据集中抽取 1000-5000 条高质量指令数据作为训练集，按 9:1 比例划分训练集与验证集，测试集可自行从原数据集中抽取不参与训练的 200 条数据用于最终效果评估。



2. 代码与环境依赖

核心依赖库: Hugging Face Transformers、PEFT、BitsAndBytes、Accelerate、Datasets、PyTorch
官方参考仓库: QLoRA 原始论文官方实现 <https://github.com/artidoro/qlora>
基座模型要求: 推荐使用开源的 7B 及以下参数量大语言模型, 降低硬件门槛, 例如 Qwen2-7B-Instruct、Llama-2-7B-Chat、ChatGLM3-6B、Phi-3-mini-4K
硬件提示: 对于硬件条件有限的同学, 可使用 AutoDL、Colab、Kaggle 等云计算平台完成训练, 单卡 10G 及以上显存即可完成全流程实验。

三、核心理论理解要求

在动手实践前, 需通过论文与源码阅读, 完成以下核心理论的梳理, 并在最终报告中完成完整阐述, 这是本项目的核心考核内容之一:

1. QLoRA 核心原理与创新点

- ① 解释 QLoRA 的核心设计思路, 为什么 QLoRA 能在大幅降低显存占用的同时, 保留接近全参数微调的模型效果。
- ② 对比 QLoRA 与标准 LoRA、全参数微调的核心差异, 从显存占用、训练效率、模型效果、部署成本四个维度进行对比分析。
- ③ 解释 LoRA 低秩适配的底层原理: 低秩矩阵分解的核心逻辑, 为什么仅更新少量适配器权重, 就能让大模型适配下游任务, 而无需更新全量参数。

2. LoRA 关键超参数与选型逻辑

- ① 说明 LoRA 核心超参数 (rank、lora_alpha、lora_dropout、target_modules) 的作用, 以及不同参数设置对模型容量、训练效果、过拟合风险的影响。
- ② 解释大模型指令微调中, target_modules 的选型依据, 为什么不同基座模型需要选择不同的目标模块进行微调。

3. 大模型指令微调核心逻辑

- ① 说明有监督微调 (SFT) 的训练目标与损失函数计算逻辑, 解释为什么需要对 Prompt 模板部分进行掩码处理, 不计入损失计算。
- ② 阐述指令数据集的构建核心原则, 说明高质量指令数据对模型微调效果的关键影响。
- ③ 解释大模型的 Prompt 模板匹配逻辑, 为什么微调时必须使用与基座模型一致的 Prompt

模板，模板不匹配会导致哪些问题。

四、任务要求与最低实现标准

本项目重点考察全流程的打通与原理理解，而非单纯追求模型效果指标，完成以下要求即达到考核基线：

（1）环境配置与模型加载

- ① 正确配置 QLoRA 微调环境。
- ② 严禁直接使用第三方一键式微调框架（如 LLaMA-Factory）的封装脚本完成全流程，必须手动实现核心的模型加载、QLoRA 配置、训练循环与推理逻辑，框架仅可作为原理参考。

（2）数据预处理

- ① 自行编写数据处理脚本，完成指令数据集的清洗、去重、长度过滤、格式校验，转换为基座模型对应的 Prompt 对话格式。
- ② 完成数据集的划分与 DataLoader 封装，确保训练集、验证集、测试集无数据泄露，编写数据格式校验脚本，保障输入数据与模型要求的格式完全匹配。

（3）QLoRA 微调训练

- ① 基于 PEFT 库实现 QLoRA 适配器的配置与挂载，完成有监督微调（SFT）训练脚本的编写，支持训练过程中验证集评估、Loss 日志记录、最优权重自动保存。
- ② 训练过程中要求有清晰的日志输出（使用 logging 库实现），记录每轮训练的训练 Loss、验证 Loss、学习率变化，训练日志以 txt 格式持久化保存。
- ③ 适配单卡 GPU 训练，通过梯度检查点、梯度累积等优化策略，解决低显存环境下的训练问题，确保训练过程稳定无崩溃。

（4）模型推理与效果验证

- ① 编写推理脚本，支持 LoRA 权重的加载与合并，实现微调后模型的批量推理与交互式推理。
- ② 设计至少 5 类不同维度的测试用例（知识问答、文案生成、逻辑推理、信息提取、场景化指令），每类不少于 4 条测试指令，完成微调前后基座模型的输出效果对比。
- ③ 实现基础的效果评估，定性分析模型在指令遵循能力、生成质量、上下文理解上的变化，统计模型的指令响应准确率。

五、提交内容

1.可运行代码：包含数据处理、模型加载、QLoRA 微调、训练验证、推理测试、部署脚本

本的完整项目文件夹，代码需有清晰注释，标注核心实现逻辑与关键参数修改位置。

2.模型权重：训练得到的最优 LoRA 适配器权重文件（无需提交完整基座模型权重），以及权重加载、合并的详细说明文档。

3.训练过程文件：完整的训练日志（txt 格式）、训练 / 验证 Loss 下降曲线可视化结果、训练关键超参数配置说明文档。

4.效果验证材料：

(1) 微调前后模型在测试用例上的回答对比表格，包含指令输入、基座模型原始输出、微调后模型输出、效果评价三个核心列。

(2) 至少 5 组交互式推理的对话记录，完整体现微调后模型的指令遵循能力与场景适配效果。

(3) 若完成部署相关加分项，需提供可视化界面截图、接口调用演示结果与部署说明。

5. 项目说明报告：

(1) 完整解答第三部分中所有核心理论理解要求的问题，阐述对 QLoRA 与大模型微调的核心原理理解。

(2) 数据集选用说明、数据预处理全流程、子集规模与划分方式，附数据格式示例。

(3) 训练环境与核心配置说明：基座模型选型、量化方案、LoRA 关键超参数、训练硬件与环境依赖版本。

(4) 微调全流程实现说明，训练过程中遇到的核心问题与对应的解决方法。

(5) 模型效果定性与定量分析，消融实验结果（如有），误差分析（模型回答错误 / 效果不佳的案例，以及对应的原因分析）。

(6) 本次项目的学习过程、个人思考与总结，对大模型轻量化微调技术的个人理解。

附加项目三：BLIP 官方代码跑通中文图像描述

一、考核目的

1. 理解 BLIP 核心思想与统一架构

在多模态大模型（MLLM）的发展进程中，如何高效对齐“视觉”与“语言”两大模态是核心挑战。本考核题旨在全面考察对现代统一视觉语言理解与生成模型全流程的理解与动手能力。重点要求掌握 BLIP 提出的统一多模态混合编码器-解码器（MED）架构，并能够动手实践其在图像字幕生成（Image Captioning）与 图文检索（Image-Text Retrieval）两个核心下游任务中的灵活迁移能力。

论文链接：<https://arxiv.org/abs/2201.12086>

2. 用官方 BLIP 代码在中文数据上跑通

基于 Salesforce 官方仓库 [salesforce/BLIP](https://github.com/salesforce/BLIP), 跑通图文检索与图像字幕两个核心任务的微调与推理流程: 用 Flickr30k 的图像, 配对 Flickr30k-CN 的机器翻译中文字幕, 同时产出中文图像字幕和实现中文章-图双向检索。无需算法创新。

允许只用小规模子集(例如 1,000 张图)完成作业, 重点是打通两个不同任务的数据 → 训练 → 推理的完整流程, 不以指标为导向。

二、数据与代码

请自行下载数据集, 并进行相关处理。推荐使用 Flickr8k 或 Flickr30k, 也可自行寻找其他开源中文图像字幕数据集。

1.代码: salesforce/BLIP 官方实现。克隆并用 `pip install -r requirements.txt` 安装依赖。

<https://github.com/salesforce/BLIP>

2.图像: Flickr30k (仅用图像即可; 注意遵守许可, 可下载原图或“可分发版本(含图片链接 + 英文字幕)”)。

(Shannon Lab)<https://shannon.cs.illinois.edu/DenotationGraph/data/index.html>

3.字幕: Flickr30k-CN (机器翻译得到的中文字幕, 带 train/val/test 划分)。

三、理论理解

在动手实践前, 需通过论文与源码阅读, 完成以下几个方面的理论梳理:

1. 深入理解 Q-Former 的“信息瓶颈”设计

Q-Former 是连接冻结视觉编码器与冻结 LLM 的灵魂桥梁。请在报告中详细回答:

①Query 的作用: 为什么 BLIP-2 不直接将 ViT 的全部 Patch Tokens 输入给语言模型? 引入可学习的 Learnable Queries 是为了解决什么问题?

②双层注意力机制: 说明 Q-Former 内部的 Self-Attention (Query 之间交互) 与 Cross-Attention (Query 与图像特征交互) 是如何协同工作的。

③任务解耦: 简述在第一阶段预训练中, Q-Former 是如何通过三种损失函数 (ITC, ITM, ITG) 强制 Queries 吸收与文本相关的视觉信息的。

2. BLIP-2 整体架构与组件理解

请围绕 BLIP-2 架构、组件作用、训练思想、模块化设计进行完整阐述:

(1) BLIP-2 定位、角色与核心组成

请在报告中写明：

- ①说明 BLIP-2 是什么，它在多模态大模型中的角色是什么，主要解决什么问题？
- ②列出 BLIP-2 的三大核心组件，并写出每类组件常见的具体模型示例。
- ③简述 BLIP-2 的两大核心设计思想。

(2) 各组件作用、整体工作流程与设计优势

请逐条回答：

- ①视觉编码器的作用是什么？
- ②Q-Former 的桥梁作用体现在哪三方面？
- ③LLM 的作用是什么？
- ④对比传统端到端多模态模型，写出 BLIP-2 的优势。

(3) 扩展理解：模块化设计、通用性与应用场景

- ①请完成模块化设计分析：说明视觉模块、Q-Former 桥梁模块、LLM 语言模块如何解耦，为何可以像 API 一样独立替换、升级。
- ②列出典型下游任务与应用。

3. BLIP 的统一架构 (MED) 与任务切换

理解同一个模型如何通过 任务特定的掩码 (Task-specific Masking) 实现功能转换：

理解模式：学习在进行“图文检索”时，模型如何利用双向自注意力 (Bi-directional Self-attention) 提取全局特征。

生成模式：学习在进行“字幕生成”时，模型如何切换为因果自注意力 (Causal Self-attention) 来逐字预测文本。

4. 中文化改造的工程逻辑

针对中文任务，需明确说明：

- ①词表替换：为什么必须将 bert-base-uncased 替换为 bert-base-chinese？
- ②参数对齐：替换 Tokenizer 后，Q-Former 的文本端 Embedding 层和最后的 Linear 投影层需要做哪些维度检查？

四、最低要求

1.明白 BLIP-2 通过 Vision Encoder → Q-Former → LLM 三段式架构实现视觉与语言模态的对齐，能够在不进行训练的情况下，直接基于预训练权重完成图像问答推理；理解 Q-Former 中 Query Tokens 在跨模态信息桥接中的关键作用。本实验重点在于跑通模态对

齐的推理流程，而非从头训练。（在云端跑）

2.环境配置与模型加载

```
conda create -n blip2 python=3.10 -y
```

```
conda activate blip2
```

```
pip install torch torchvision torchaudio
```

```
pip install transformers accelerate pillow matplotlib scikit-learn
```

（以上依赖来自 HuggingFace 官方 BLIP-2 模型卡。）

(HuggingFace)<https://huggingface.co/Salesforce/blip2-opt-2.7b>

（1）加载预训练权重。

- ① 基于 HuggingFace 加载 BLIP-2 预训练模型，推荐使用 Salesforce/blip2-opt-2.7b。
- ② 正确加载 Blip2Processor 与 Blip2ForConditionalGeneration。
- ③ 将模型迁移至 GPU（如可用），并设置为推理模式（eval）。

（2）验证模型加载是否成功。

- ① 任选一张测试图像 + 简单 Prompt，确认模型能输出合理文本。
- ② 检查显存占用与推理耗时，确保后续脚本可正常运行。

3.数据预处理（满足小子集 1,000 张图即可）

必须使用给定的数据集，同学自行编写脚本，完成图像加载、文本预处理与 DataLoader 封装。

（1）编写 prepare_flickr30k_cn.py，将原始 Flickr30k-CN 数据转换为 COCO 格式 JSON。

- ① JSON 格式要求：包含 "images" 和 "annotations" 两个字段，每条标注需包含 "image_id" 和 "caption"（中文）。
- ② 从 Flickr30k 取约 1,000 张图片（随便抽样，记录随机种子）。
- ③ 从 Flickr30k-CN 取与之匹配的中文字幕（沿用其官方 train/val/test split）。
- ④ 对齐键：图片文件名或图片 ID，注意处理大小写和扩展名不一致。

（2）让 BLIP 的 DataLoader 读取这个 JSON。

- ① 将生成的 JSON 命名为 coco_karpathy_train.json（和官方一致），放入 ann_root 指定目录。
- ② 修改 configs/caption_coco.yaml 中的 image_root 和 ann_root 路径，使其指向你的图片和标注目录。

4.实现交互式问答函数

（1）功能要求。

- ① 支持用户输入图片路径与自然语言问题。
- ② 调用预训练 BLIP-2 模型完成推理，并输出生成答案。
- ③ 支持循环交互，可输入退出指令（如 q）终止程序。
- ④ 对异常输入（如路径错误、空问题）进行基本的容错处理。

(2) 运行交互式问答命令：

```
python interactive_vqa.py
--model_name Salesforce/blip2-opt-2.7b
```

注：使用预训练权重直接推理即可，无需训练；本任务重点在于跑通和理解模态对齐的推理流程。

5.Q-Former 中间表征提取与可视化展示

(1) 原理理解：在动手前，需理解 BLIP-2 的 Q-Former 结构与 Query Tokens 机制，明白其如何将视觉特征压缩为可被 LLM 理解的语义查询表示。

(2) 运行特征提取命令：

以交互式问答脚本为基础，确保其能正确调用 vision_model 与 qformer 模块。

修改 extract_qformer.py 中的 image_path、output_dir 路径和相关参数。

运行命令：

```
bash python extract_qformer.py \ --image_path ./data/test.jpg --output_dir ./output/qformer_feat
```

(3) 可视化展示要求：

- ① 原图展示 + Q-Former Query 相似度热力图：将输入原图与 32 个 Query Token 之间的相似度矩阵可视化，输出文件命名为 qformer-queries-sim.png。
- ② Query 向量 PCA 投影：对 Q-Former 输出的 Query 向量进行 PCA 降维，绘制二维投影散点图并标注 token 编号，输出文件命名为 qformer-queries-pca.png。
- ③ 不同问题的回答对比：针对同一张图片，输入 3~5 个不同问题，记录模型生成的答案，并以表格形式输出对比结果（包含图片 ID、问题、模型回答三列）。

6.流程梳理与总结

(1) 描述一张图片输入 BLIP-2 模型后，最终得到文本回答的完整数据处理流程。请按顺序梳理关键步骤（无需涉及复杂数学公式）。

- ① 图像预处理：将原始图片读取并转换为统一尺寸与 RGB 格式的张量输入。
- ② 视觉编码：经 Vision Encoder (ViT) 提取图像特征，得到视觉 token 序列。
- ③ Q-Former 模态对齐：通过一组可学习的 Query Tokens 与视觉特征交互，输出固定数量（如 32 个）的语义查询向量。
- ④ 跨模态投影：将 Q-Former 输出映射到 LLM 的输入空间，与文本 Prompt 拼接。
- ⑤ 语言模型生成：LLM 基于拼接后的输入序列进行自回归生成，输出文本 token。
- ⑥ 文本解码：使用 Tokenizer 将生成的 token 序列解码为最终自然语言回答。

(2) 要求以文字 + 流程图（可手绘或使用绘图工具）形式呈现，体现各模块之间的输入输出关系。

五、提交内容

1.可运行代码：包含数据处理、模型定义、训练、推理的完整代码，需有清晰注释；

- 2.模型权重：

(1) 字幕生成任务训练得到的最优模型权重文件。

(2) 图文检索任务训练得到的最优模型权重文件。

3.训练日志：两个任务分别的训练日志，包含 Loss 下降曲线、评估指标等关键训练过程记录；

4.可视化结果：

- (1) 字幕生成：至少 5 张测试图片及其生成的字幕，图文并排展示。
- (2) 图文检索：至少 2 组文本检索图像样例和 2 组图像检索文本样例，清晰展示输入与检索结果。

5.简短报告：

- (1) 使用的数据子集规模与对齐方法；
- (2) 选用的中文 BERT 模型名称与参数；
- (3) 训练配置：分别列出字幕生成与图文检索两个任务的 `batch_size / epoch / learning_rate` 等关键配置。
- (4) 可视化样例：汇总展示两个任务的可视化结果。
- (5) 误差分析：
 - ① 字幕生成：分析 1-2 个生成错误（如重复、错词、实体遗漏）的案例。
 - ② 图文检索：分析 1-2 个检索失败的案例（如将猫检索成狗，或将精细描述匹配到错误图片），并尝试解释可能的原因。